MCP Database Copilot - Technical Architecture

Document Version: 1.0
Date: July 2026

Table of Contents

Overview

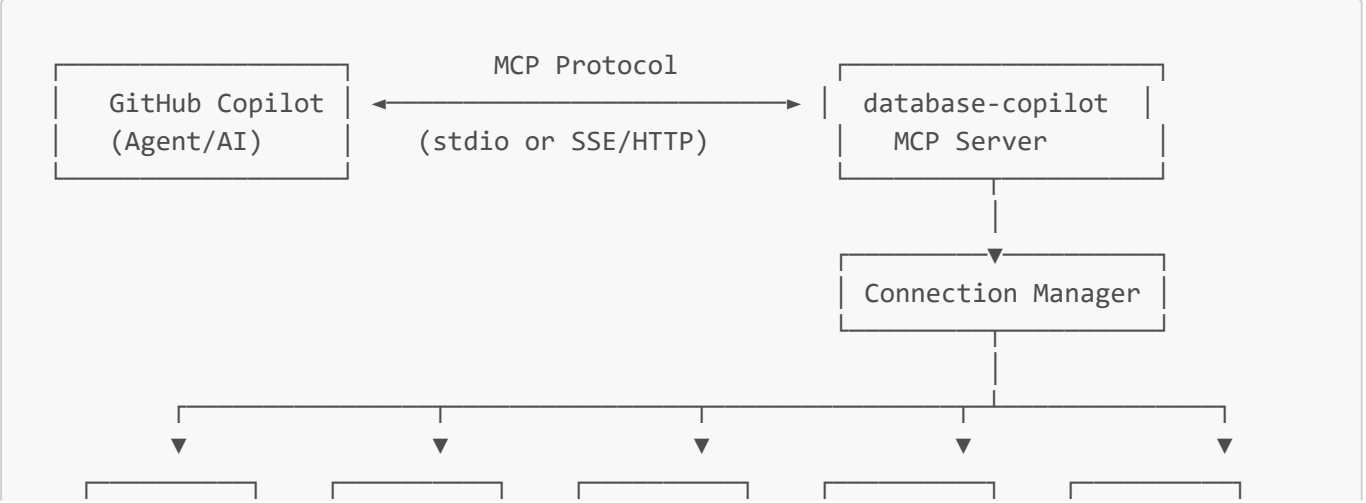
MCP Database Copilot is a **Model Context Protocol (MCP) server** that connects AI assistants (GitHub Copilot, Claude, etc.) directly to databases. It exposes real database schema, relationships, indexes, and sample data so AI can write accurate queries against actual tables.

Key Features

- **Schema Exploration** — List schemas, tables, columns with types, constraints, and comments
- **Relationship Mapping** — Foreign key discovery with Mermaid ER diagrams
- **Index Analysis** — View existing indexes and get suggestions for missing ones
- **Safe Query Execution** — Read-only queries with automatic LIMIT enforcement
- **Query Plan Analysis** — EXPLAIN output for performance optimization
- **Multi-Database Support** — Connect to multiple databases simultaneously

---

Architecture Diagram



```
    │PostgreSQL│     │  MySQL   │     │  SQLite  │     │ MongoDB  │     │   CSV    │
    │ Adapter  │     │ Adapter  │     │ Adapter  │     │ Adapter  │     │ Adapter  │
    └──────────┘     └──────────┘     └──────────┘     └──────────┘     └──────────┘
```

MCP Communication Protocol

The Model Context Protocol (MCP) is a JSON-RPC-like protocol for AI-tool communication.

Transport Options

Transport	Description	Use Case
stdio	Communication via stdin/stdout pipes	VS Code direct integration
SSE	Server-Sent Events over HTTP	HTTP-based clients

Message Format

Request (Agent → Server):

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "list_tables",
    "arguments": {
      "schema": "public",
      "connectionId": "postgresql://localhost:5432/mydb"
    }
  }
}
```

Response (Server → Agent):

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "content": [{
      "type": "text",
      "text": "Tables in mydb.public:\n  📊 users (~1000 rows)\n  📊 orders
(~5000 rows)"
    }]
  }
}
```

HTTP API Endpoints

When running in SSE mode, the server exposes 3 HTTP endpoints:

Method	Endpoint	Description
`GET`	`/sse`	Opens Server-Sent Events connection for MCP protocol
`POST`	`/messages?sessionId=xxx`	Receives tool call requests from the agent
`GET`	`/health`	Health check endpoint

Endpoint Details

GET /sse

Opens a persistent SSE connection. Returns a `sessionId` used for subsequent requests.

POST /messages

Receives MCP tool calls. Requires `sessionId` query parameter.

Request Body:

```
{
  "method": "tools/call",
  "params": {
    "name": "execute_query",
    "arguments": { "sql": "SELECT * FROM users LIMIT 10" }
  }
}
```

GET /health

Returns server status.

Response:

```
{
  "status": "ok",
  "server": "database-copilot",
  "version": "2.0.0"
}
```

Available MCP Tools

Schema Tools

Tool	Description	Parameters
`list_connections`	List all configured database connections	—
`list_schemas`	List schemas/databases in a connection	`connectionId?`
`list_tables`	List tables/views/collections with row counts	`schema?`, `connectionId?`
`describe_table`	Get column details (types, constraints)	`table`, `schema?`, `connectionId?`
`full_table_info`	Columns + FKs + indexes all at once	`table`, `schema?`, `connectionId?`
`search_schema`	Search for tables/columns by pattern	`pattern`, `connectionId?`

Relationship Tools

Tool	Description	Parameters
`get_relationships`	Get foreign key relationships	`table?`, `schema?`, `connectionId?`
`schema_overview`	High-level ER diagram (Mermaid)	`schema?`, `connectionId?`

Index Tools

Tool	Description	Parameters
`get_indexes`	List indexes on a table	`table`, `schema?`, `connectionId?`
`suggest_indexes`	Suggest missing indexes	`table`, `schema?`, `connectionId?`

Query Tools

Tool	Description	Parameters
`execute_query`	Run read-only queries (SQL/JSON)	`sql`, `limit?`, `connectionId?`
`explain_query`	Get execution plan for a query	`sql`, `connectionId?`
`sample_data`	Get sample rows from a table	`table`, `schema?`, `limit?`, `connectionId?`
`suggest_query`	Get schema context for query formulation	`description`, `connectionId?`

Database Adapters

Supported Databases

Database	Adapter	Type	Default Port
PostgreSQL	`PostgresAdapter`	Relational	5432
MySQL	`MySQLAdapter`	Relational	3306
MariaDB	`MariaDBAdapter`	Relational	3306
SQL Server	`SQLServerAdapter`	Relational	1433

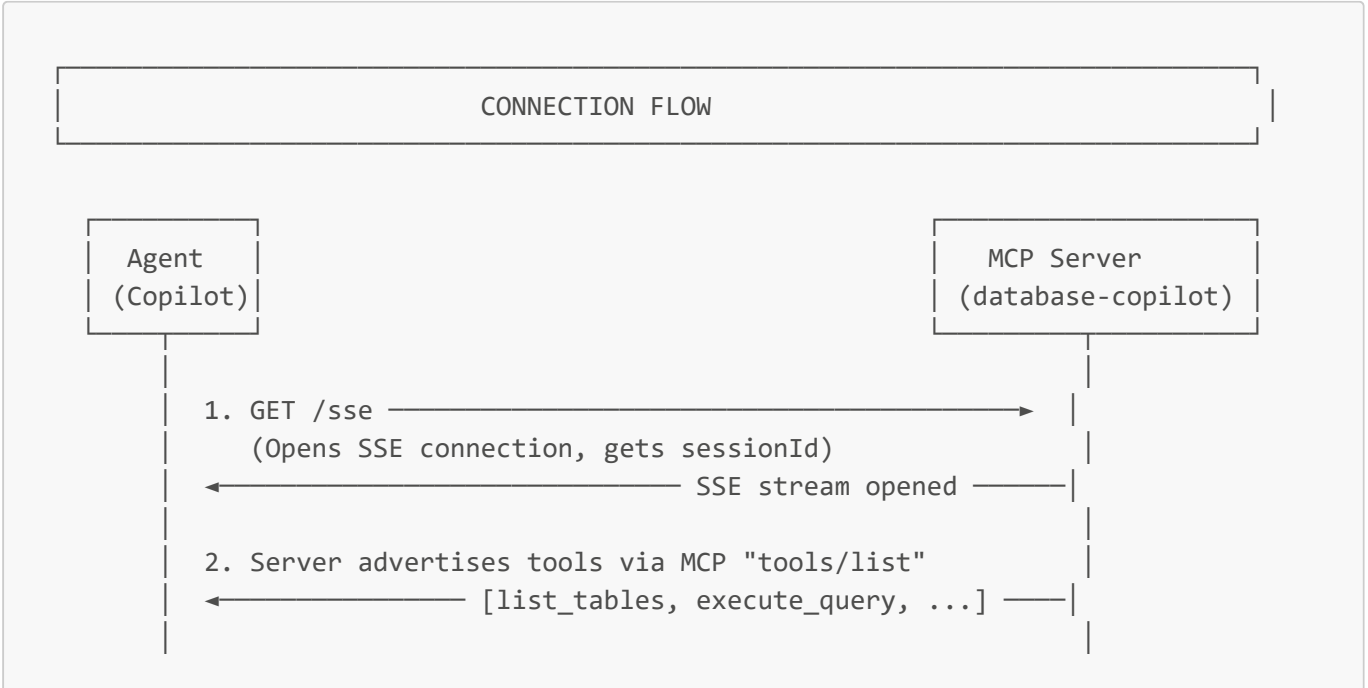
Database	Adapter	Type	Default Port
SQLite	`SQLiteAdapter`	Embedded	—
MongoDB	`MongoDBAdapter`	Document	27017
CSV	`CSVAdapter`	File-based	—

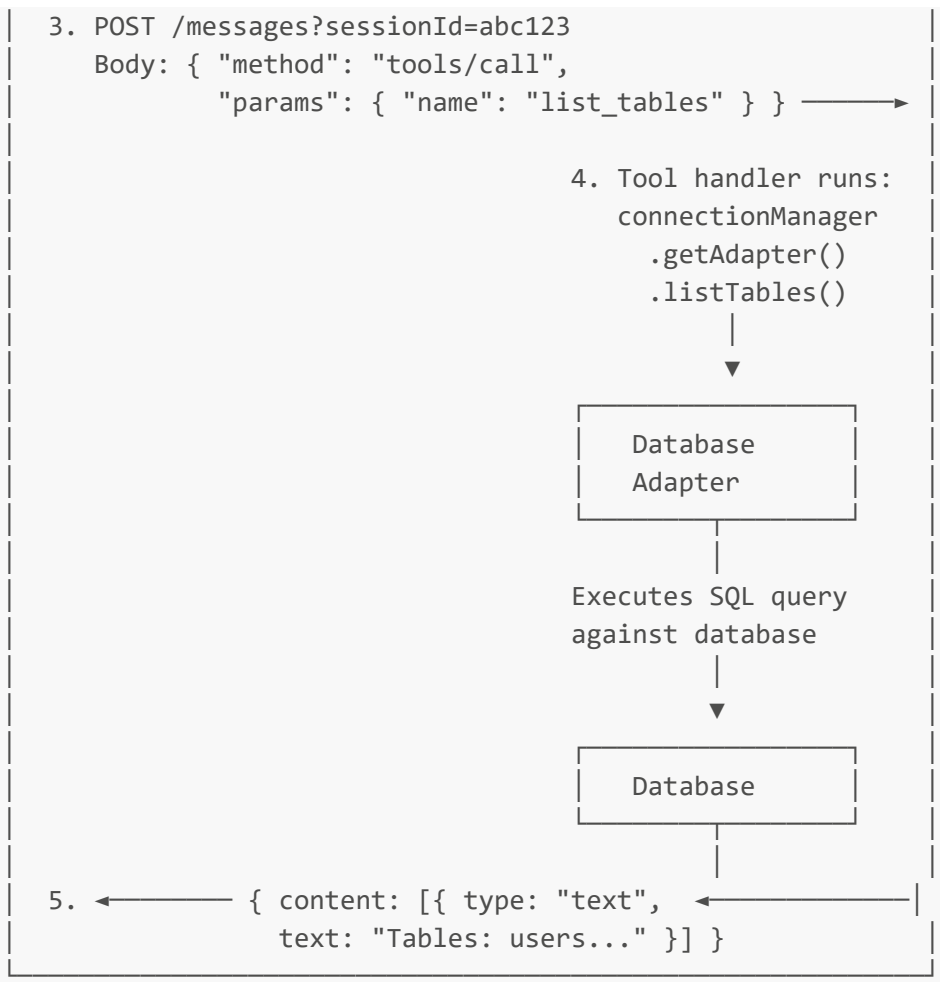
Adapter Interface

All adapters implement the `BaseAdapter` interface:

Method	Description
`connect()`	Establish database connection
`disconnect()`	Close connection
`listSchemas()`	Get available schemas
`listTables(schema)`	Get tables with row counts
`getColumns(table, schema)`	Column metadata
`getForeignKeys(table, schema)`	Foreign key relationships
`getIndexes(table, schema)`	Index information
`executeQuery(sql, limit)`	Run read-only queries
`explainQuery(sql)`	Get execution plan
`getSampleData(table, schema, limit)`	Sample rows
`searchColumns(pattern)`	Search columns by pattern

Connection Flow





Connection Selection Logic

1. If `connectionId` is provided → use that specific connection
2. Otherwise → use the **default** (first configured) connection

Connection ID Format: `type://host:port/database`
Example: `postgresql://localhost:5432/mydb`

Configuration

VS Code Integration (mcp.json)

```
{
  "servers": {
    "database-copilot": {
      "type": "stdio",
      "command": "node",
      "args": ["${workspaceFolder}/src/index.js", "--transport", "stdio"],
      "env": {
        "DATABASE_URL": "postgresql://user:pass@localhost:5432/mydb"
      }
    }
```

```
    }
  }
```

Environment Variables

Single Database (DATABASE_URL):

```
DATABASE_URL=postgresql://user:pass@localhost:5432/mydb
```

Multiple Databases (DB_x_ pattern):

```
DB_1_TYPE=postgresql
DB_1_HOST=localhost
DB_1_PORT=5432
DB_1_DATABASE=app_db
DB_1_USER=myuser
DB_1_PASSWORD=mypass

DB_2_TYPE=mongodb
DB_2_HOST=mongo.example.com
DB_2_PORT=27017
DB_2_DATABASE=analytics
```

Server Configuration

Variable	Description	Default
`MCP_TRANSPORT`	Transport type (`stdio` or `sse`)	`sse`
`MCP_PORT`	HTTP port for SSE mode	`3100`

File Structure

```
database-copilot/
├── src/
│   ├── index.js              # Entry point
│   ├── server.js             # MCP server setup & HTTP endpoints
│   ├── config.js             # Environment variable parsing
│   ├── database/
│   │   ├── base-adapter.js   # Abstract adapter interface
│   │   ├── connection-manager.js  # Connection registry
│   │   └── adapters/
│   │       ├── index.js      # Adapter registry
│   │       ├── postgresql.js
│   │       ├── mysql.js
│   │       ├── mariadb.js
```

```
│   │       ├── sqlserver.js
│   │       ├── sqlite.js
│   │       ├── mongodb.js
│   │       └── csv.js
│   └── tools/
│       ├── schema.js         # Schema exploration tools
│       ├── relationships.js  # FK and ER diagram tools
│       ├── query.js          # Query execution tools
│       └── indexes.js        # Index tools
├── docs/
│   ├── SETUP.md
│   └── VSCODE_INTEGRATION.md
└── package.json
```

Generated from MCP Database Copilot source code analysis